2019ICTS140

1. Project Overview

This project is a cloud-native Todo application developed for the Cloud Application Development course (2019ICTS140). The application consists of a Node.js/Express backend with a simple frontend that allows users to create, view, and delete to do items. The application uses a JSON file for data persistence and is fully containerized using Docker for easy deployment on cloud platforms like Google Cloud Run.

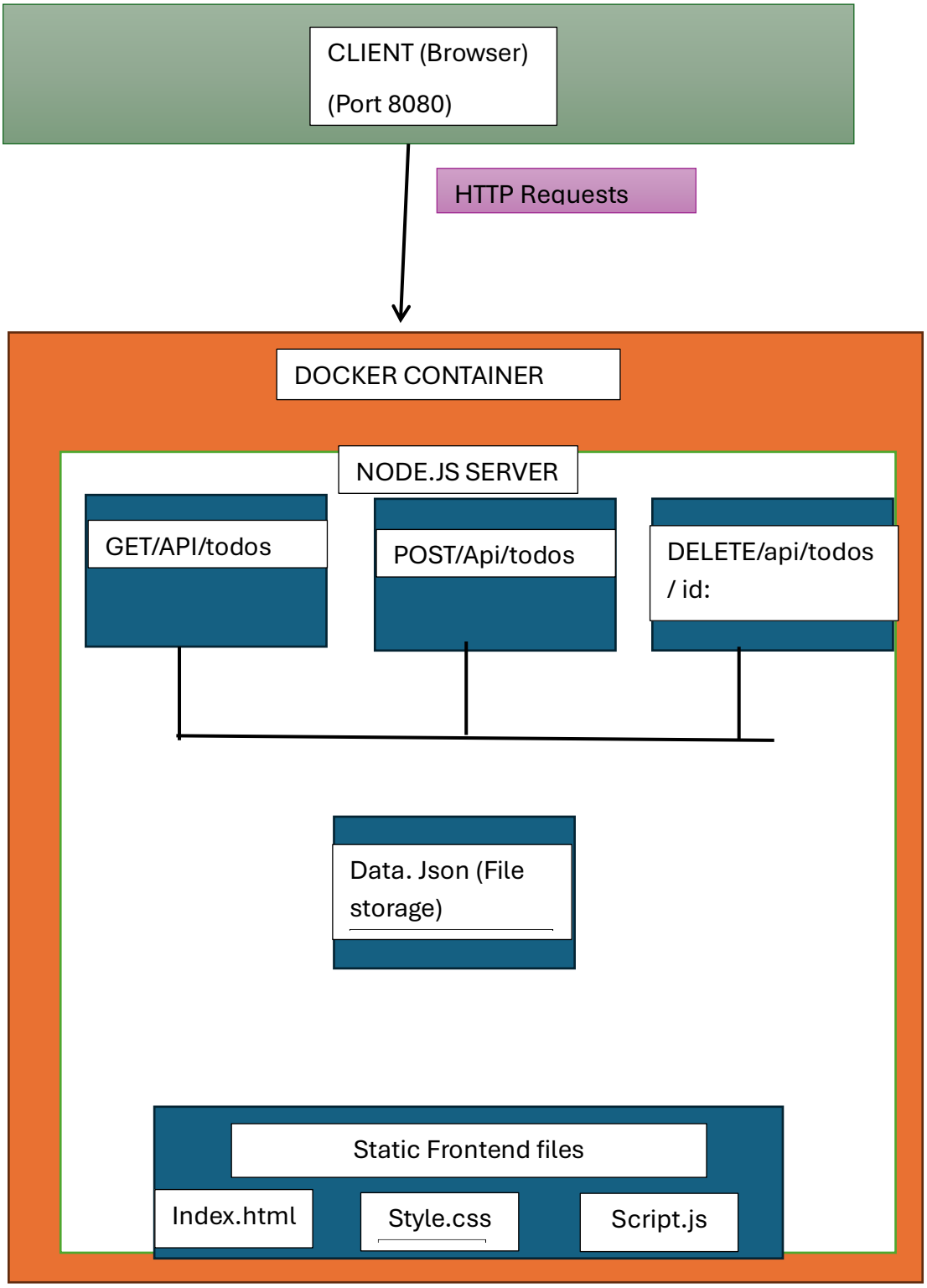
Technologies Used:

- **Backend:** Node.js with Express.js
- **Frontend:** HTML, CSS, JavaScript
- **Containerization:** Docker
- **Database:** JSON file storage
- **Port:** 8080 (cloud-ready)

Key Features:

- RESTful API endpoints for to do operations
- Responsive frontend with Google Fonts
- CORS enabled for cross origin requests
- Environment variable configuration
- Docker containerization for portability

2. Architecture Diagram



Component Interactions:

1. Browser requests the frontend (served statically)
2. Frontend JavaScript makes API calls to the backend
3. Express routes handle CRUD operations
4. Data is read/written to JSON file
5. All components run inside a single Docker container
6. Container exposes port 8080 for external access

3. Docker file Explanation

```
# Use Node.js base image
FROM node:18-alpine

# Set working directory
WORKDIR /usr/src/app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install --production

# Copy application source code
COPY . .
```

```
# Expose the port (GCP Cloud Run expects 8080)
EXPOSE 8080

# Command to run the application
CMD [ "node", "server.js" ]
```

Optimizations Used:

- Multi-stage not needed (simple app)
- Alpine base for smaller image
- Production-only dependencies
- Layer caching by copying package files first

4. Deployment Steps

Step 1: Build Docker Image

```
docker build -t nilofaa/2019icts140:v1 .
```

Step 2: Run Container Locally

```
docker run -p 8080:8080 nilofaa/2019icts140:v1
```

Access at: http://localhost:8080

Step 3: Test the Application

- Open browser and verify frontend loads
- Test adding todos

- Test deleting todos
- Verify data persists in container

Step 4: Push to Docker Hub

```
docker login
docker push nilofaa/2019icts140:v1
```

Step 5: Deploy to Google Cloud Run (Optional)

```
# Tag for Google Container Registry
docker tag nilofaa/2019icts140:v1 gcr.io/[PROJECT-ID]/2019icts140:v1

# Push to GCR
docker push gcr.io/[PROJECT-ID]/2019icts140:v1

# Deploy to Cloud Run
gcloud run deploy 2019icts140 \
  --image gcr.io/[PROJECT-ID]/2019icts140:v1 \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --port 8080
```

Step 6: Verify Deployment

- Get the Cloud Run URL
- Test all functionality

- Monitor logs: gcloud run logs read 2019icts140

5. ISSUES FACED AND SOLUTIONS

- Challenge: Race Conditions during Container Startup.

Solution: Implemented a 'pg_isready' healthcheck in Docker Compose and a retry mechanism/delayed initialization strategy in the Node.js 'initDb' function to handle database latency.

- Challenge: Package.json Entry Point Mismatch

Problem**:** package.json had "main": "index.js" but actual entry point was server.js.

Solution**:** Either renamed file or added start script:

- Challenge: Static Asset Delivery.

Solution: Leveraged 'express.static' to serve the frontend assets directly from the Node.js process, reducing the need for a separate Nginx container for simple deployments.

6. CONCLUSION

The project successfully demonstrates the integration of a Node.js backend within a containerized ecosystem. By following these architectural patterns and deployment strategies, the application achieves high availability and scalability, making it a solid foundation for more complex cloud based systems.